AnalysisTools user manual

RNDr. Karel Šindelka, Ph.D.
k.sindelka@gmail.com

Version 4.0 (Released ~~August 18, 2020~~ at some point, it will be...)

Contents

1 Introdution **2**
1.1 Installation 2

2 Format of input/output files **3**
2.1 Structure files 3
2.1.1 VTF format 3
2.1.2 FIELD format 4
2.1.3 XYZ format 5
2.1.4 LAMMPS data format 5
2.1.5 LAMMPS lammpstrj format 5
2.2 Coordinate files 6
2.2.1 VTF format 6
2.2.2 XYZ format 6
2.2.3 LAMMPS lammpstrj format 7
2.3 Aggregate file 7

3 Utilities **9**
3.1 AddRandomToSystem 10
3.2 Aggregates and Aggregates-NotSameBeads 12
3.3 Average 13
3.3.1 Estimate autocorrelation time via -tau option 14
3.3.2 Block averages via -b option 15
3.3.3 Moving averages via -m option 15
3.4 DensityBox 16
3.5 Info 17
3.6 Selected 18

1. Introdution

AnalysisTools is a set of utilities to (mainly) analyse trajectories of particle-based simulations. The 34 utilities (including one from the DL_MESO simulation package can be roughly divided into four types:

- utilities to calculate system-wide properties; e.g., pair correlation functions or particle densities along a simulation box axis
- utilities to calculate per-molecule or per-aggregate properties (where aggregate stands for any supramolecular structure); e.g., shape descriptors for individual molecules or whole micelles
- utilities to manipulate a configuration; e.g., create initial configuration for a simulation from scratch or by adding molecules to an existing one
- helper utilities to analyse text files; e.g., calculate averages and standard deviations of a data series

The `Examples` directory contains examples showcasing capabilities of individual utilities as well as possible workflows chaining several utilities.

1.1 Installation

AnalysisTools requires `C` and `Fortran` compilers and `cmake`. The utilities should be compiled in a separate directory, typically `<root>/build` (where `<root>` is the AnalysisTools root directory). The following generates a `Makefile` within `<root>/build`:

```
mkdir <root>/build; cd <root>/build; cmake ../
```

To then compile the utilities, simply run `make` (to compile all utilities) or `make <utilitiy name>` (to compile a single utility) in `<root>/build`. The binaries are located in the `<root>/build/bin` directory.

2. Format of input/output files

This section describes several file types used by many of the AnalysisTools utilities. Output files for the utilities themselves are described in their respective sections in Chapter 3.

Input files are divided into two categories: structure and coordinate files. The structure files contain information about the system composition, that is, number of beads and molecules, bead properties (mass, charge, etc.), molecule properties (number and types of beads, connectivity, etc.). The coordinate files contain individual timesteps with beads' coordinates (and, possibly, velocities, forces, etc.). Some files can be used as both a structure and a coordinate file, but using a different structure file may be essential. For example, an `xyz` file (see below) may be used as both, but because it only provides bead names, using a separate structure file would provide additional information (bead and/or molecule properties).

For working with aggregates, AnalysisTools' `agg` file format (see below) describes their compositions in terms of molecule numbers from a structure file used to generate it. Any of the `Aggregates` (3.2) utilities can create the `agg` file.

2.1 Structure files

As structure files, any of the following formats can be used; example files for a simple 'sandbox' system are provided in the `Examples/InputFiles` directory. Utilities load a specific structure using the `-i <file>` option; if the option is missing, it is usually assumed the structure file is the same as the coordinate file.

`Info` utility (Section 3.5) can convert these formats between each other via the `-o <file>` option.

2.1.1 VTF format

See here for complete description and `in.vtf` and `in.vsf` for example files (a `vtf` file contains a structure part followed by a coordinate part, but a `vsf` file contains only the structure part). In this format, every bead is defined on its own line and all bonds all listed.

Not all keywords listed on the web page for the 'Atom lines' are recognized; keyword `n[ame]` is mandatory and optional keywords are `m[ass]`, `charge|q`, `r[adius]`,

resid, and res[name]. For the <aid-specifier>, only a single bead index number or the default keyword can be used. Other keywords are ignored.

AnalysisTools groups beads (and molecules) into bead (and molecule) types. Generally, only name defines types of beads, i.e., should two beads share the name, they will be of the same type; their mass, charge, and radius will each equal to that for the bead type's topmost atom line containing the corresponding keyword. If the keyword is missing from all atom lines, that characteristic is undefined. The --detailed command line option triggers detailed recognition, where beads that share the name but differ in mass, charge, and/or radius will be of different type.

Different molecule types, on the other hand, are always distinguished based on all their characteristics, i.e., name, connectivity, and order of beads (order of its vsf indices). However, molecule types can also be affect by the --detailed option because of the bead type definition

For the effect of the --detailed option, see the Examples/Info directory.

For bond lines, only single bonds are recognized (i.e., bond <index1>:<index2> format).

Periodic boundary condition can be included as pbc <x> <y> <z> for a cuboid box or pbc <a> <b> <c> <alpha> <beta> <gamma> for a general paralleloid box.

2.1.2 FIELD format

A format similar to the FIELD input file for DL_MESO_DPD simulation package; see in.FIELD for an example. This concise format defines species of beads and molecules, providing number of beads/molecules of given species as well as their properties.

The species section is the same as in the DL_MESO FIELD file, but the molecules section differs slightly. Besides the beads part, every molecule can have bonds, angles, and dihedrals, but even though up to three parameters for potentials are read, AnalysisTools does not recognize the potential keywords (harm, cos, etc.). These potential parameters are only used for generating new structure files via the Info utility (TODO: reference). The molecules section can also contain one extra part: impropers for improper angles which has the same format as the dihedrals section (this was added because the LAMMPS molecular dynamics simulator distinguishes between the two).

Note that the FIELD file does not have to contain Interactions section; this part is ignored.

To be recognized by AnalysisTools utilities, the file must either be called FIELD or have a .FIELD extension.

2.1.3 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description and `in.xyz` for an example. An `xyz` file is essentially a file with timesteps (bead coordinates), and the only structure information are bead names.

The comment line at the beginning of each timestep can contain periodic boundary conditions as `<x> <y> <z>` (cuboid box) or `<a> <b> <c> <alpha> <beta> <gamma>` (general paralleloid box); it is similar to the `pbc` keyword in `vtf` file. To use this box definition, use `-pbc <n>` option where `<n>` is the position of the first number on the comment line (e.g., `1` if the comment line starts with the box size).

2.1.4 LAMMPS data format

This rather complicated format comes from the LAMMPS molecular dynamics simulator and lists the numbers of beads, bonds, angles, etc., as well as individual beads, bonds, angles, etc., and information about potentials for bonds, angles, etc. See here for description of the format and `in.data` for an example.

AnalysisTools reads the header information as well as `Masses`, `Atoms`, `Velocities`, `Bonds`, `Angles`, `Dihedrals`, `Impropers`. It also reads `Bond Coeffs`, `Angle Coeffs`, `Dihedral Coeffs`, and `Improper Coeffs` sections.

The `Atoms` section must have `atom_style full` format, that is every bead line must have the format `<id> <mol id> <bead type> <q> <x> <y> <z>`, where `<id>` is a bead index, `<mol id>` is a molecule index (-1 means the bead is in no molecule), `<bead type>` is the bead type index corresponding to a line in the `Masses` section, `<q>` is charge, and `<x> <y> <z>` are coordinates.

For the `Bond Coeffs` and `Angle Coeffs` sections, harmonic potential is assumed, and the first value of each bond/angle type is multiplied by 2 because LAMMPS uses harmonic spring strength of $k/2$ (as opposed to, e.g., DL_MESO which uses k).

For the `Dihedral Coeffs` and `Improper Coeffs` sections, AnalysisTools reads at most three numbers without assuming any specific potential.

All the `Coeffs` sections are used only for generating new structure files via the `Info` utility, TODO: reference (specifically, only to generate `FIELD` and LAMMPS `data` files because no other supported file contains this information).

To be recognized by AnalysisTools utilities, the file must have an extension `.data`.

2.1.5 LAMMPS lammpstrj format

This mainly coordinate file format comes from the LAMMPS dump style custom command. See `in.lammpstrj` for an example.

Of the possible LAMMPS attributes, AnalysisTools recognizes only `id` (bead index – mandatory), `element` (bead name), `x y z` (bead coordinates), `vx vy vz` (bead velocities), and `fx fy fz` (bead forces). Any other attribute is ignored. Note that if `element` is missing, all beads are considered of the same type called `bt`.

To be recognized by AnalysisTools utilities, the file must have an extension `.lammpstrj`.

2.2 Coordinate files

As coordinate files, any of the following formats can be used; example files for a simple 'sandbox' system are provided in the `Examples/InputFiles` directory. These files can be combined with any of the above-described structure files (via the `-i <file>` option), but some may be used as both the structure and coordinate file.

`SelectedVcf` utility (TODO: reference) can generate any of these file types, converting between the formats; all utilities that require coordinate file can use any of these formats.

2.2.1 VTF format

See here for the description of a coordinate part of the `vtf` format and `in.vtf` and `in.vcf` for example files (a `vtf` file contains the coordinate part after a structure part, and a `vcf` file contains only the coordinate part).

AnalysisTools recognizes both ordered and indexed timesteps as well as a `pbc` line defining box dimensions.

If an AnalysisTools utility is not provided with an extra structure file (`-i <file>` option), the same filename is assumed but with the `.vsf` extension instead of the `.vcf` extension.

2.2.2 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description and `in.xyz` for an example.

The comment line at the beginning of each timestep can contain periodic boundary conditions as `<x> <y> <z>` (cuboid box) or `<a> <b> <c> <alpha> <beta> <gamma>` (general paralleloid box); it is similar to the `pbc` keyword in `vtf` file. To use this box definition, use `-pbc <n>` option where `<n>` is the position of the first number on the comment line (e.g., `1` if the comment line starts with the box size).

The `xyz` coordinate file will be used as a structure file as well unless an extra structure file is provided via the `-i <file>` option.

2.2.3 LAMMPS lammpstrj format

This coordinate file format comes from the LAMMPS dump style custom command. See `in.lammpstrj` for an example.

Of the possible LAMMPS attributes, AnalysisTools recognizes only `id` (bead index – mandatory), `element` (bead name), `x y z` (bead coordinates), `vx vy vz` (bead velocities), and `fx fy fz` (bead forces). Any other attribute is ignored. Note that if `element` is missing, all beads are considered of the same type called `bt`.

To be recognized by AnalysisTools utilities, the file must have an extension `.lammpstrj`.

This coordinate file will be used as a structure file as well unless an extra structure file is provided via the `-i <file>` option.

2.3 Aggregate file

A `file.agg` is generated using `Aggregates` (or `Aggregates-NotSameBeads`) utility. The file contains information about the number of aggregates in each timestep and which molecules and monomeric (i.e., unbonded) beads belong to which aggregate. It serves as an additional input file for utilities that calculate aggregate properties; `agg` file is, therefore, linked to the `vcf` file that was used to generate it.

The `agg` file is a simple text file. The first two lines are just comments (the second one should contain the command used to generate the file as parts fo the command may be used by subsequent aggregate analysis). From third line, the data for individual timesteps are shown. It follows these rules:

- each timestep starts with `Step: <int>`
- the second line is the number of aggregates in the given timestep
- for each aggregate, there is a single line in the format `<size> : <id1> <id2> ... <idsize>`; the `<id#>` are molecular indices from the input structure file
- no blank or comment lines are allowed
- not all molecules present in the `vcf` file used to generate this file must be present in every timestep

Note that the term aggregate also refers to free chains (i.e., fully dissolved chains).

When keywords `Last Step: <int>` are encountered in place of `Step:` line, it signalizes the end of the `agg` file; no utility will read anything beyond this line.

If `vtf` format is used for the coordinates, the indices from `agg` file can be used in vmd to visualize, e.g., only a specific aggregate by using `resid <id1> ... <id#>` in the `Selected Atoms` box inside the vmd. A bash script `scripts/VisualizeAgg.sh` (and the accompanied picture VisualizeAgg.jpg) is a rough example of visualizing aggregates by different colours (it takes `vtf` and `agg` files as command line arguments).

An example of an `agg` file can be found in the `Examples/DistrAgg` directory.

3. Utilities

All utilities have command line options affecting their behaviour. The following options are common for many utilities, so they are described here rather than at the individual utilities where only list of the possible options is provided.

General options	
`--verbose`	print information about all bead and molecule types to the screen
`--silent`	run silently, i.e., without any output at all (overrides `--verbose` option)
`--help`	print short description of the utility and usage
`--version`	print version of the utilities and exit
Options for structure and coordinate input files	
`-i <name>`	use specified structure file, see Section 2.1 for possible formats
`--detailed`	only `vtf` structure file: detailed recognition for bead (and, consequently, molecule) types based on the bead names as well as charges, masses, and radii
`-pbc <n>`	only `xyz` file: position of the first box dimension in the file's comment line at the beginning of each timestep
`-st <n>`	any coordinate file: start calculations at `n`-th timestep
`-e <n>`	any coordinate file: end calculations at `n`-th timestep
`-sk <n>`	any coordinate file: skip every `n` timesteps during calculations

For `vtf` structure files, the bead types are defined solely based on their name by default; the other properties (mass, charge, and radius) are taken from the first `a[tom]` line with the bead of that name that contains the appropriate keyword. For example, lines

```
atom default name A m 1
atom 1 name B
atom 2 name B m 2
atom 3 name A m 2
atom 4 name A
```

would define two bead type called `A` and `B` with masses equal to 1 and 2, respectively. Using the `--detailed` option would split the `A` beads into three types: `A` and `A_1`

with masses equal to 1 and 2, respectively, and `A_2` with undefined mass (it remains undefined because there is an ambiguity—should the last bead have mass 1 or 2?). On the other hand, the `B` beads would be assigned the same bead type with mass equal to 2 (there is no ambiguity because only one mass is specified for the `B` beads). See `Examples/Info` directory for other examples.

Plain `xyz` format provides only bead names and coordinates, but the comment line at the beginning of every timestep can store extra information. Here, only box dimensions are read, but because there are extended `xyz` formats containing more information, the `-pbc` option is used to specify where the box dimensions are (as well as to tell the utility there are box dimensions at all). For example, `-pbc 3` must be specified for the following comment line

`other text 10 10 10 90 60 30`.

This would define a paralleloid box with $a = b = c = 10$ and angles $\alpha = 90°$, $\beta = 60°$, and $\gamma = 30°$. If less than 6 numbers were provided, only the first three would be used to define a cuboid box.

Which timesteps are used by the utility can be controlled via the `-st`, `-e`, and `-sk` options. For example, specifying `-st 2 -e 10 -sk 2` would use timesteps 2, 5, and 8.

3.1 AddRandomToSystem

This utility takes an existing system specified by `vtf` file(s) and adds new beads into it, placing them either randomly or using provided coordinates. The utility requires an input coordinate file containing all beads (i.e., all beads defined in the structure file must be present in the coordinate). If an incomplete coordinate file is provided, `AddRandomToSystem` exhibits undefined behaviour, i.e., it can run without problems, exit with an error, or freeze.

The utility reads what is to be added from a `FIELD`-like file, placing the new beads and molecules randomly. The random coordinates of added beads can be constrained via several options; for added molecules, either the molecule's first bead (default behaviour) or its geometric centre (`--gc` option) obey the constraints. The coordinates of the remaining beads in the molecule are governed by the coordinates in the `FIELD`-like file. Therefore, not all the molecular beads necessarily obey the constraining rules. Molecules are added with a random orientation.

There are two types of constraints which can be combined: place new beads (i) specified distance from other beads or (ii) in a specified interval in x, y, and/or z directions. In (i), options `-ld` and/or `-hd` specify the distance; if present, these must be accompanied by `-bt` option. Then, the new beads are placed at least `-ld <float>` and at most `-hd <float>` distance from beads specified by the `-bt` option. In (ii), options `-cx`, `-cy`, and `-cz` basically change the box size for the added beads;

for example, `-cx 5 9` would generate x coordinates only in the interval $\langle 5, 9)$. All these options can be combined, but note that `AddRandomToSystem` does not perform any sanity checks, that is, if any combination of the provided options is impossible to achieve, the utility will run forever.

As stated above, the structure and number of added molecules and monomeric beads are read from a `FIELD`-like file. This file must contain a `species` section followed by a `molecule` section as described in the DL_MESO simulation package. Following is a short description of the two sections.

The new unbonded beads appear in the resulting files after the unbonded beads from the original system (but before any bonded beads). If molecules are added, `AddRandomToSystem` places them at the end.

By default, the new beads are added to the system, increasing the total number of beads. However, if `--switch` is used, the new beads replace beads in the original system. The replaced beads are either neutral beads with the lowest indices (as per ordering in the provided structure file) or beads of the type specified using `-xb` option. In both cases, the beads to be replaced must be unbonded.

The size of the new simulation box can be changed using the `-b` option. There are no constraints on the size of the cuboid box; if the new box is small, not all beads from the original system are necesserily inside the box (although the added once always are). The origin of the new box coincides with the origin of the original box unless `--centre` option is used; in that case, the centres of the two boxes coincide.

`AddRandomToSystem` creates `vcf` and `vsf` files containing the new system; the coordinates can also be written into an `xyz` file.

Several examples of using the utility are provided in the `Examples/AddRandomToSystem` directory.

Usage: `AddToSystem <input.vcf> <out.vsf> <out.vcf> <options>`

Mandatory arguments	
`<input.vcf>`	input coordinate file (either `vcf` or `vtf` format)
`<out.vsf>`	output `vsf` structure file for the new system
`<out.vcf>`	output `vcf` coordinate file for the new system

Non-standard options	
`-st <int>`	`<input.vcf>` timestep to add new beads to (default: 1)
`-xyz <name>`	also save coordinates to `xyz` file
`--switch`	replace original beads instead of increasing the total number of beads
`-b <x> <y> <z>`	side lengths of the new simulation box

`--centre`	place the original simulation box in the middle of the new one
	Random placement
`-f <name>`	`FIELD`-like file specifying additions to the system (default: `FIELD`)
`-ld <float>`	lowest distance from beads specified by `-bt` option
`-hd <float>`	highest distance from beads specified by `-bt` option
`-bt <bead names>`	bead types to use in conjunction with `-ld` and/or `-hd` options
`-cx <num> <num2>`	constrain x coordinate to interval $\langle num, num2)$
`-cy <num> <num2>`	constrain y coordinate to interval $\langle num, num2)$
`-cz <num> <num2>`	constrain z coordinate to interval $\langle num, num2)$
`--gc`	use molecule's geometric centre for distance check
	Provided coordinates
`-vtf <vsf> <vcf>`	add ready-made system from `vtf` files instead of randomly placing beads read from the `FIELD`-like file

3.2 Aggregates and Aggregates-NotSameBeads

These utilities determine which molecules belong to which aggregates (note that 'aggregate' is used for any supramolecular structure) according to a simple criterion: two molecules belong to the same aggregate if they share at least a specified number of contact pairs. A contact pair is a pair of two beads belonging to different molecules which are closer than a specified distance. The information is written in `.agg` text file described in Section 2.3.

Periodic boundary conditions can be removed from whole aggregates via the `-j` option; the aggregate's centre of mass is always inside the simulation box. This is useful for visualization (an aggregate will no longer be split by the walls of the simulation box, but it may stretch far outside the boundaries of the box) as well as for further analysis (the other utilities do not have to join the aggregates, possibly speeding up the analysis).

While the `Aggregates` utility uses all possible pairs of given bead types, `Aggregates-NotSameBeads` does not use same-type pairs. For example, if bead types `A` and `B` are given, `Aggregates` will use all three possible bead type pairs (i.e., `A-A`, `A-B`, and `B-B`), but `Aggregates-NotSameBeads` will use only `A-B` bead type pairs. Fig 3.1 illustrates the behaviour: using `Aggregates ... A B` would find one contact pair between molecules 1 and 2 and two contacts between molecules 2 and 3, whereas `Aggregates-NotSameBeads ... A B` would ignore A-A and B-B pairs, finding no contact pair between molecules 1 and 2 and only one between molecules 2 and 3.

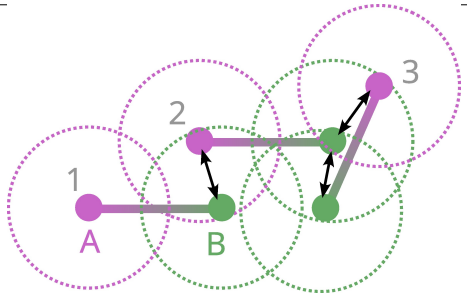

Figure 3.1: Simple example of distance checks—three two-bead molecules, where dotted lines represent maximum pair distance (`-d` option) and black arrows show bead pairs.

Note that when `-st`, `-e`, or `-sk` options are used, the resulting `agg` file is no longer coupled to the original coordinate file (i.e., they cannot be used together for further analysis), but it is coupled to the optional output file with joined coordinates generated via the `-j` option.

Usage: `Aggregates <input> <out.agg> <bead type(s)> [options]`

Mandatory arguments	
`<input>`	input coordinate file
`<out.agg>`	output `agg` file
Options	
`-d`	maximum distance for contact (default: 1)
`-c`	minimum number of contacts (default: 1)
`-j <file>`	save coordinates of joined aggregate to a coordinate file
Other options (see the beginning of Chapter 3)	
`-st`, `-e`, `-sk`, `-i`, `-pbc`, `--detailed`, `--verbose`, `--silent`, `--help`, `--version`	

3.3 Average

This utility calculates one of three types of averages based on a used option from specified column(s) of data in the input text file (all `#`-starting lines and blank lines are ignored). It either produces an overall average with statistical error and an autocorrelation time estimate (`-tau` option), block averages (`-b` option), or moving averages (`-m` option). While the `-tau` option appends a single line to the output file, either of the `-b` or `-m` options creates a new output file with somewhat smoother data.

The first and last lines used for the average calculation can be controlled using the standard `-st` and `-e` options.

3.3.1 Estimate autocorrelation time via -tau option

The average value of an observable $\mathcal{O}$ is a simple arithmetic mean,

$$\langle \mathcal{O} \rangle = \frac{1}{N} \sum_{i=1}^{N} \mathcal{O}_i, \tag{3.1}$$

where N is the number of measurements and the subscript i denotes individual measurements. If the measurements are independent (i.e., uncorrelated), the statistical error, ϵ, is given by:

$$\epsilon^2 = \frac{\sigma^2_{\mathcal{O}_i}}{N}, \tag{3.2}$$

where $\sigma^2_{\mathcal{O}_i}$ is the variance of the individual measurements,

$$\sigma^2_{\mathcal{O}_i} = \frac{1}{N-1} \sum_{i=1}^{N} (\mathcal{O}_i - \langle \mathcal{O} \rangle)^2. \tag{3.3}$$

For correlated data, the autocorrelation time, τ, representing the number of steps between two uncorrelated measurements must be determined. Every τ-th measurement is uncorrelated, so the equation (3.2) can then be used to estimate the error.

A commonly used method to estimate τ is the binning (or block) method. In this method, the correlated data are divided into N_B non-overlapping blocks of size k ($N = kN_\mathrm{B}$) with per-block averages, $\mathcal{O}_{\mathrm{B},n}$, defined as:

$$\mathcal{O}_{\mathrm{B},n} = \frac{1}{k} \sum_{\substack{i=1+\\(n-1)k}}^{kn} \mathcal{O}_i. \tag{3.4}$$

If $k \gg \tau$, the blocks are assumed to be uncorrelated and equation (3.2) can be used:

$$\epsilon^2 = \frac{\sigma^2_\mathrm{B}}{N_\mathrm{B}} = \frac{1}{N_\mathrm{B}(N_\mathrm{B}-1)} \sum_{n=1}^{N_\mathrm{B}} (\mathcal{O}_{\mathrm{B},n} - \overline{\mathcal{O}})^2. \tag{3.5}$$

An estimate of the autocorrelation time can be obtained using the following formula:

$$\tau_\mathcal{O} = \frac{k\sigma^2_\mathrm{B}}{2\sigma^2_{\mathcal{O}_i}}. \tag{3.6}$$

The number of blocks, N_B, is supplied as an argument of the `-tau` option and Average then appends a single line to the `<output>` file; the line starts with N_B and continues with three values ($\langle\mathcal{O}\rangle$, ϵ, and $\tau_\mathcal{O}$) per every data column specified in the Average command.

A way to quickly get a τ estimate is to use a wide range of N_B values and plot $\tau_\mathcal{O}$ from equation (3.6) as a function of N_B. Because the number of data points in one block should be significantly larger than the autocorrelation time (e.g., ten times larger), plotting $f(x) = N/(10x)$ will produce a monotonously decreasing curve that intersects the $\tau_\mathcal{O}$ vs. N_B curve. A value of $\tau_\mathcal{O}$ near the intersection (but to the left, where the decreasing curve is above $\tau_\mathcal{O}$ vs. N_B curve) can be considered a safe estimate of τ.

3.3.2 Block averages via -b option

Besides estimating the autocorrelation time, the per-block averages can be themselves plotted to get (probably) smoother dataset. Using the `-b` option, the number of datapoints per block to average, k, is supplied, and the utility prints the per-block averages from equation (3.4) to the output file.

This way of averaging could be useful for example with density data produced by DensityBox or related utilities; if the bin width supplied to the DensityBox was too small, it is (possibly much) faster to block-average the densities rather than rerun DensityBox. For this case, specify the first column (distance) along with any density columns from the density file.

3.3.3 Moving averages via -m option

More common way to smoothing noisy data is to use the moving (or rolling or running) average (or moving mean or rolling mean); this common method does have many names.

Similarly to the block-average, the first element of the moving average is a simple mean of k values. Unlike with block-average, however, the k values for the next element are obtained by ignoring only one value and taking the next k values; i.e., $k-1$ values from the previous element of the moving average are always reused as opposed to the block-average where the blocks of data are not overlapping.

The moving average elements, $\mathcal{O}_{\mathrm{M},n}$, are defined as

$$\mathcal{O}_{\mathrm{M},n} = \frac{1}{k}\sum_{n}^{n+k-1}\mathcal{O}_i. \tag{3.7}$$

Usage: `Average <input> <output> <column(s)> <options>`

Mandatory arguments	
`<input>`	input text file
`<output>`	output text file
`<column(s)>`	at least one column number from `<input>`

Options	
`-tau <int>`	τ estimation where `<int>` represents the number of blocks N_B
`-b <int>`	block-average printing mode where `<int>` represents the number of datapoints per one block, k (equation (3.4))
`-m <int>`	moving average mode where `<int>` represents the number of datapoints per one moving block, k (equation (3.7))
Other options (see the beginning of Chapter 3)	
`-st`, `-e`, `--verbose`, `--silent`, `--help`, `--version`	

3.4 DensityBox

This utility calculates number density for all bead types along all three axes directions of the simulation box, generating one file per axis. The density is calculated from 0 to box length in the given direction, that is, the box is 'sliced' into blocks with width `<width>` (see below), and numbers of different bead types are counted in each 'slice'. Or to put it more rigorously, a density profile for a bead type i along an axis α, $\rho(\alpha)$, is calculated as

$$\rho_i(\alpha) = \frac{\sum_j \delta(\alpha - \alpha_j^i)}{\Delta\alpha L_\beta L_\gamma}, \tag{3.8}$$

where $\delta(\alpha - \alpha_i^j)$ gives the number of i beads inside a slice of the simulation box of the thickness $\Delta\alpha$ along the α-axis; L_β and L_γ represent box side lengths along the two remaining axes.

The utility does not distinguish between beads with the same name in different molecules, so if one bead type is in more than one molecule type, its density will be averaged over all molecule types it appears in. If one requires densities specific to certain molecules containing the same bead types, the `-x` option can be used to first run the utility without one molecule type and then rerun it, excluding the other molecule type. Thus, two output files (per axis) are generated, each missing densities from the specified molecule types.

Note that this utility assumes orthogonal box with constant side lengths; in case of triclinic box and/or varying box size, undefined behaviour may occur (i.e., the utility crashes), and any results will not be reliable.

Also note that the same density profile (along with other properties) can also be calculated via the Local utility (**??**).

Usage: `DensityBox <input> <width> <output> <options>`

Mandatory arguments	
`<input>`	input coordinate file (either `vcf` or `vtf` format)
`<width>`	width of each bin of the distribution
`<output>`	three output files with automatic `-x.rho`, `-y.rho`, and `-x.rho` endings
Non-standard options	
`-x <mol name(s)>`	exclude specified molecule type(s) (i.e., do not calculate density for beads in molecules `<mol name(s)>`)
Other options (see the beginning of Chapter 3)	
`-st`, `-e`, `-sk`, `-i`, `--help`, `-pbc`, `--detailed`, `--verbose`, `--silent`, `--version`	

Format of output files:
1) `<output>` – bead densities
- first line: command used to generate the file
- second line: column headers
  - first is the centre of each bin (governed by `<width>`); i.e., if `<width>` is 0.1, then the centre of bin 0 to 0.1 is 0.05, centre of bin 0.1 to 0.2 is 0.15, etc.
  - the rest are for the calculated data: each column corresponds to the number density of the specified bead type

3.5 Info

This utility prints the system information read from an input structure file, possibly augmenting it with information from another structure file (`-i[!] <file>` option) and/or pruning the system to contain only beads from a provided coordinate file (`-c` option). For the effects of these options, see the `Examples/Info` folder.

`Info` can also write this information into a new structure file of the given format (`-o` option). Allowed output files are `vtf` structure file (with the extension `.vsf`), LAMMPS `data` file (either with the extension `.data` or a filename without any recognizable extension), and DL_MESO FIELD file (called `FIELD` or with the extension `.FIELD`). Coordinates are also printed to output LAMMPS `data` or `vtf` file. These coordinates are 0 if no coordinate file is specified or the input structure file does not contain them. Note that if no coordinate file is specified, coordinates

of all beads are written (e.g., even when only some beads are specified in an input `vtf` file; unspecified beads have coordinates 0).

There are a few options for the output structure file. For LAMMPS `data` file, `--mass` can be used to specify atom types by their mass, but print different charges in the `Atoms` section (i.e., `Info`-recognized bead types with the same mass are aggregated into the same LAMMPS atom types). Furthermore, extra atom types (with mass 1) can be added via the `-ebt` option; these atoms do not appear in the `Atoms` section of the `data` file and can be used as, e.g., extra atom type when srp potential (i.e., non-crossing bonds created by adding ghost particles between bonded beads) is used. For `vtf` /`vsf` file, `atom default` bead can be specified via the `-def` option; by default, the bead type with the most unbonded beads is used as the `atom default`.

Usage: `Info <input> <options>`

Mandatory argument	
`<input>`	input structure file
Options for input files	
`-i[!] <file>`	extra input structure file; see `Examples/Info` directory for the effect of `-i` and `-i!` options (default: None)
`-c <file>`	input coordinate file (default: None)
Options for output file	
`-o <file>`	output coordinate file (default: None)
`-def <bead>`	only `vtf` structure file: use bead type `<bead>` for the 'atom default' line (default:bead type with the most unbonded beads)
`--mass`	only `data` structure file: define LAMMPS atom types by mass but print per-atom charges in the `Atoms` section (default: atom types are the same as `Info`-recognized bead types)
`-ebt <int>`	only `data` structure file: number of extra atom types (for, e.g., srp)
Other options (see the beginning of Chapter 3)	
`-st`, `-pbc`, `--detailed`, `--verbose`, `--silent`, `--help`, `--version`	

3.6 Selected

This utility creates a new coordinate file of the `xyz` , `vcf` , or `lammpstrj` format. By default, the utility saves all bead and molecules types, but using `-bt` and/or `-mt` option specifies which bead and/or molecule types to exclude from the output file.

If `--reverse` option is used, the specified bead and/or molecule types are instead the only ones that are saved.

Besides the standard `-st`, `-e`, and `-sk` options, which timesteps to save can be explicitly specified via the `-n` option that can take a maximum of 100 arguments (the `-st`, `-e`, and `-sk` are then ignored). Also, using `--last` option saves only the last step from the input coordinate file (all the previous options are ignored if `--last` is used). If lammps `data` file is used as a coordinate file, all the above options are ignored as the `data` file contains by definition only a single timestep.

There is also an option to remove periodic boundary conditions for molecules (i.e., to join them) via `--join` switch. Conversely, the simulation box can be wrapped (i.e., the periodic boundary conditions applied, putting all beads inside the box) via `--wrap` switch. If both `--wrap` and `--join` options are used, the simulation box is first wrapped and then the molecules are joined.

Usage: `Selected <input> <output> <bead type(s)> <options>`

Mandatory arguments	
`<input>`	input coordinate file
`<output>`	output coordinate file

Options	
`-bt <bead type>`	bead types to exclude
`-mt <mol type>`	molecule types to exclude
`--reverse`	save only specified types instead of excluding them
`--join`	join molecules by removing periodic boundary conditions
`--wrap`	wrap simulation box (i.e., apply periodic boundary conditions)
`-n <int(s)>`	save only specified timesteps
`--last`	save only the last step

Other options (see the beginning of Chapter 3)
`-st`, `-e`, `-sk`, `-i`, `-pbc`, `--detailed`, `--verbose`, `--silent`, `--help`, `--version`